MEGA ASSESSMENT EXAMINATION (45 QUESTIONS)

Subject: C Programming | Topics: Functions, Function Calls, Recursion

Instructions: Contains 15 questions per topic. Formatted strictly to GATE and PSU standards.

Section A: Functions, Scope, and Storage Classes

Q1. What is the output of a function returning a local 'auto' variable's address?

```
int* foo() { int a = 10; return &a; }
```

(A) 10
(B) Compilation Error
(C) Runtime Error / Dangling Pointer
(D) 0

Q2. Which storage class restricts the use of the address-of (&) operator?

(A) auto
(B) register
(C) static
(D) extern

Q3. What is the output?

```
int x = 5;
void print() { printf("%d ", x); }
int main() { int x = 10; print(); return 0; }
```

(A) 5
(B) 10
(C) Garbage
(D) Error

Q4. What is the default return type of a function in C89 if not explicitly specified?

(A) void
(B) int
(C) char
(D) double

Q5. A static function in C is:

(A) Visible to all files
(B) Restricted to the file where it is declared
(C) Allocated in heap
(D) Faster to execute

Q6. What happens if a function signature is 'void foo(void)' but it is called with an argument?

(A) Compiles fine, argument ignored
(B) Compile-time error
(C) Runtime error
(D) Undefined behavior

Q7. Consider the return statement: return (a, b, c); What does it return?

(A) a
(B) b
(C) c
(D) Array of a,b,c

Q8. Can the main() function in C be called recursively?

(A) Yes, C allows it
(B) No, it's the entry point only
(C) Yes, but only once
(D) Causes compilation error

Q9. What is the output?

```
void foo() { static int i = 5; if(--i){ printf("%d ", i); foo(); } }
```

(A) 4 3 2 1
(B) 5 4 3 2
(C) 4 3 2 1 0
(D) Infinite loop

Q10. How do you declare a variable length argument function?

(A) void foo(int a, ...)
(B) void foo(...)
(C) void foo(int a, *)
(D) void foo(args)

Q11. Which keyword hints the compiler to substitute the function body at the call site?

(A) macro
(B) substitute
(C) inline
(D) fast

Q12. What is the output?

```
int main() {
  extern int a;
  printf("%d", a);
  return 0;
}
int a = 20;
```

(A) 0
(B) 20
(C) Error
(D) Garbage

Q13. Nested functions in standard C are:

(A) Fully supported
(B) Supported via pointers only
(C) Not supported by ISO C
(D) Same as inline functions

Q14. The atexit() function is used to:

(A) Terminate the program
(B) Register a function to be called at normal program termination
(C) Handle exceptions
(D) Free memory immediately

Q15. Shadowing occurs when:

(A) Two functions have the same name
(B) A local variable has the same name as a global or outer block variable
(C) A pointer hides a variable
(D) Recursion goes too deep

Section B: Function Calls, Parameter Passing, and Evaluation

Q16. C passes arguments to functions by:

(A) Value only
(B) Reference only
(C) Both Value and Reference natively
(D) Pointer references only

Q17. When an array is passed to a function, what is actually passed?

(A) Entire array copy
(B) Address of the first element
(C) Size of the array
(D) Address of the last element

Q18. What does sizeof(arr) return inside a function 'void foo(int arr[])'?

(A) Total bytes of the array
(B) Size of an integer pointer
(C) Number of elements
(D) Compilation error

Q19. Evaluate the output (assuming right-to-left evaluation order):

```
int a = 1;
printf("%d %d %d", a, ++a, a++);
```

(A) 1 2 3
(B) 3 3 1
(C) 3 2 1
(D) Compiler dependent (Undefined behavior)

Q20. Which is the correct syntax for a pointer to a function returning int and taking a float?

(A) int *p(float);
(B) int (*p)(float);
(C) int p*(float);
(D) (int*) p(float);

Q21. How do you pass a struct to a function to avoid copying overhead?

(A) Pass by value
(B) Pass a pointer to the struct
(C) Pass struct elements individually
(D) Use an inline struct

Q22. What is a callback function in C?

(A) A function that calls itself
(B) A function passed as an argument via a function pointer
(C) A function that returns a function
(D) A macro definition

Q23. What happens if you modify a string literal passed directly to a function like foo("hello")?

(A) Changes the string globally
(B) Creates a local copy
(C) Segmentation fault / Undefined behavior
(D) Compile error

Q24. If an argument is declared as 'const int *p', it means:

(A) The pointer cannot change
(B) The value pointed to cannot change
(C) Both pointer and value are constant
(D) The pointer must be initialized to NULL

Q25. What is the output?

```
void swap(int x, int y) { int t=x; x=y; y=t; }
int main() { int a=5, b=10; swap(a,b); printf("%d %d", a, b); return 0;}
```

(A) 10 5
(B) 5 10
(C) 0 0
(D) Error

Q26. Which of the following allows passing any pointer type to a function?

(A) int *
(B) char *
(C) void *
(D) Any struct pointer

Q27. Can a function return an array in C directly (e.g., int[] foo())?

(A) Yes, statically allocated
(B) Yes, dynamically allocated
(C) No, only pointers or structs containing arrays
(D) Yes, using typedef

Q28. What is the meaning of: void (*arr[10])(int);

(A) Function returning array of 10 pointers
(B) Array of 10 function pointers taking int and returning void
(C) Pointer to 10 functions
(D) Syntax error

Q29. What is a sequence point in C regarding function calls?

(A) The end of the function
(B) The evaluation of all arguments before entering the function body
(C) Return statement
(D) Semicolon inside loops

Q30. Using a macro vs an inline function for calls:

(A) Macros perform type checking, inline does not
(B) Inline performs type checking, macros do not
(C) Both are exactly the same
(D) Macros use stack memory

Section C: Recursion and Stack Frames

Q31. Every recursive function must have a:

(A) Static variable
(B) Global scope
(C) Base case
(D) Void return type

Q32. What is Tail Recursion?

(A) Recursion inside a loop
(B) When the recursive call is the last executed statement
(C) When a function calls itself twice
(D) When recursion returns a tail pointer

Q33. What does this code do?

```
int f(int n) { if(n==0) return 0; return (n%10) + f(n/10); }
```

(A) Reverses the number
(B) Sums the digits of the number
(C) Counts the digits
(D) Factorial

Q34. The time complexity of standard unoptimized recursive Fibonacci is:

(A) O(n)
(B) O(n log n)
(C) O(2^n)
(D) O(1)

Q35. Recursion uses which data structure internally?

(A) Queue
(B) Heap
(C) Linked List
(D) Stack

Q36. What is the output for f(3)?

```
void f(int n) { if(n>0) { f(n-1); printf("%d ", n); f(n-1); } }
```

(A) 1 2 1 3 1 2 1
(B) 3 2 1 1 2 3
(C) 1 2 3 2 1
(D) 3 2 1

Q37. Indirect recursion occurs when:

(A) A calls A directly
(B) A calls B, and B calls A
(C) A macro calls a function
(D) A calls A via pointer

Q38. What does this function return for x = 2, y = 3?

```
int f(int x, int y) { if(y==0) return 1; return x * f(x, y-1); }
```

(A) 6
(B) 8
(C) 9
(D) 5

Q39. What happens if the base case is never reached in recursion?

(A) Syntax error
(B) Heap overflow
(C) Stack overflow
(D) Ignored by compiler

Q40. How many recursive calls are made to solve Tower of Hanoi for n disks?

(A) 2n - 1
(B) 2^n - 1
(C) n^2
(D) n!

Q41. Output of f(5)?

```
void f(int n) { if(n==0) return; printf("%d ", n); f(n-1); }
```

(A) 1 2 3 4 5
(B) 5 4 3 2 1
(C) 5 5 5 5 5
(D) 0

Q42. Output of f(5)?

```
void f(int n) { if(n==0) return; f(n-1); printf("%d ", n); }
```

(A) 1 2 3 4 5
(B) 5 4 3 2 1
(C) Garbage
(D) Nothing

Q43. Nested recursion is defined as:

(A) Recursion inside a loop
(B) A recursive call passed as an argument to another recursive call
(C) Functions inside functions
(D) Multi-file recursion

Q44. What does this code print?

```
int f(int n) { static int i = 1; if(n>=5) return n; n = n+i; i++; return f(n); }
// Called as f(1)
```

(A) 7
(B) 5
(C) 4
(D) 6

Q45. Compared to iteration, recursion usually has:

(A) Lower memory overhead
(B) Higher memory overhead
(C) Faster execution time natively
(D) No base case requirement

DETAILED ANSWER KEY & EXPLANATIONS

Section A: Functions, Scope, and Storage Classes

Q.No	Answer	Explanation
Q1	**(C)**	Returning the address of a local auto variable leads to a dangling pointer because the variable's scope ends when the function returns.
Q2	**(B)**	The 'register' storage class suggests storing the variable in a CPU register, which does not have a memory address.
Q3	**(A)**	Lexical scoping: 'print' uses the global 'x' (5) as it doesn't see main's local 'x'.
Q4	**(B)**	In legacy C (C89/C90), functions default to returning 'int'.
Q5	**(B)**	The 'static' keyword on a function limits its linkage to internal, making it accessible only within the same translation unit (file).
Q6	**(B)**	'void' in parameters explicitly means no arguments are accepted, causing a compilation error if arguments are passed.
Q7	**(C)**	The comma operator evaluates all operands and returns the value of the rightmost operand.
Q8	**(A)**	C standard allows main() to be called recursively, unlike C++.
Q9	**(A)**	Static variable retains its value. It prints 4, 3, 2, 1 and stops when i becomes 0.
Q10	**(A)**	Requires and at least one named parameter before the ellipsis (...).
Q11	**(C)**	The 'inline' keyword suggests inlining to reduce function call overhead.
Q12	**(B)**	The extern keyword extends the visibility of 'a' defined later in the file.
Q13	**(C)**	ISO C does not support nested functions, though GCC provides them as an extension.
Q14	**(B)**	atexit() registers functions to execute automatically when main() terminates or exit() is called.
Q15	**(B)**	Shadowing is when an inner scope variable masks an outer scope variable with the same identifier.

Section B: Function Calls, Parameter Passing, and Evaluation

Q.No	Answer	Explanation
Q16	**(A)**	C strictly uses pass-by-value. Pass-by-reference is simulated by passing pointer values.
Q17	**(B)**	Arrays decay into pointers to their first element when passed to a function.
Q18	**(B)**	Since the array decays to a pointer, sizeof returns the size of the pointer (e.g., 4 or 8 bytes), not the array.
Q19	**(D)**	Modifying a variable multiple times without an intervening sequence point results in undefined behavior.
Q20	**(B)**	Parentheses around *p are required; otherwise, it declares a function returning an int pointer.
Q21	**(B)**	Passing the struct's memory address via a pointer prevents the entire structure from being copied on the stack.
Q22	**(B)**	

		A callback is executable code passed as an argument to other code, usually via function pointers in C.
Q23	**(C)**	String literals are stored in read-only memory. Attempting to modify them through a pointer leads to a segfault.
Q24	**(B)**	It is a pointer to a constant integer. The pointer itself can point elsewhere, but the target value is read-only via this pointer.
Q25	**(B)**	Pass by value means modifications inside 'swap' do not affect 'a' and 'b' in main.
Q26	**(C)**	A void pointer (void *) is a generic pointer that can hold the address of any data type.
Q27	**(C)**	C functions cannot return array types directly. They must return a pointer to the array or wrap the array in a struct.
Q28	**(B)**	It declares an array named 'arr' of 10 pointers to functions that take an int and return void.
Q29	**(B)**	There is a sequence point after the evaluation of all function arguments and the function designator, before the actual call.
Q30	**(B)**	Macros do blind text substitution, whereas inline functions evaluate arguments and perform standard type checking.

Section C: Recursion and Stack Frames

Q.No	Answer	Explanation
Q31	**(C)**	A base case is required to terminate the recursive calls and prevent infinite recursion/stack overflow.
Q32	**(B)**	Tail recursion occurs when the recursive call is the very last operation in the function, allowing compiler optimizations.
Q33	**(B)**	It continuously extracts the last digit using modulo 10 and adds it to the sum of the remaining digits.
Q34	**(C)**	It creates a binary tree of calls with overlapping subproblems, leading to exponential O(2^n) complexity.
Q35	**(D)**	Each recursive call pushes an activation record (stack frame) onto the call stack.
Q36	**(A)**	Tree recursion: f(3) -> f(2) 3 f(2). Resolving f(2) -> f(1) 2 f(1), and f(1) -> 1. Total: 1 2 1 3 1 2 1.
Q37	**(B)**	Indirect (or mutual) recursion involves two or more functions calling each other in a cycle.
Q38	**(B)**	It computes x raised to the power y. 2^3 = 8.
Q39	**(C)**	The program continuously pushes stack frames until it exhausts stack memory, crashing with a stack overflow.
Q40	**(B)**	The recurrence relation is T(n) = 2T(n-1) + 1, resolving to 2^n - 1 moves/calls.
Q41	**(B)**	The print statement executes *before* the recursive call (Pre-order logic), printing descending values.
Q42	**(A)**	The print statement executes *after* the recursive call returns (Post-order logic), printing ascending values.
Q43	**(B)**	Example: Ackermann function A(m, A(m, n)). The parameter itself is a recursive call.
Q44	**(A)**	f(1): i=1, n=2, i=2. f(2): i=2, n=4, i=3. f(4): i=3, n=7, i=4. f(7): n>=5 returns 7.

Q45	**(B)**	Recursion involves function call overhead and stack frame allocation for every call, consuming more memory.